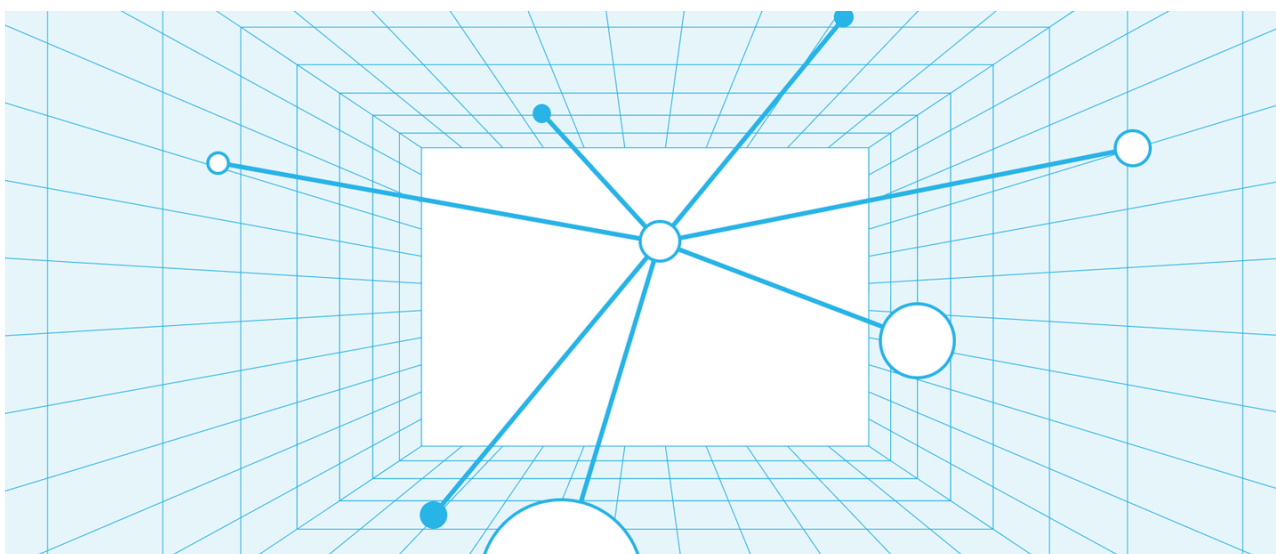


How to Choose Your Interoperable Catalog



Richie Bachala



Why the catalog choice matters now

As Apache Iceberg™ becomes the de facto open table format for modern data platforms, a critical architectural question has emerged: **Which catalog should own your Iceberg tables?**

This isn't a configuration decision you can easily reverse. The catalog that owns your table becomes the source of truth for metadata, access control, data file locations and table maintenance. It determines which engines can read and write your data, which governance policies apply and whether you're building on open standards or creating dependency on a single vendor.

For organizations running both Snowflake and Databricks, the choice typically comes down to:

- **Databricks Unity Catalog (UC):** Databricks' governance and catalog layer. UC supports both federation-based access (Lakehouse Federation via JDBC) and REST/credential-vending integrations (including for Snowflake via CLD), though supported capabilities and write semantics may differ across engines. Databricks open sourced a subset of Unity Catalog in 2024. The managed offering — what customers actually run in production — does not expose a standard Iceberg REST Catalog endpoint for external engines. The existence of an open source version of UC does not necessarily address the interoperability considerations of the managed offering.
- **Snowflake Horizon Catalog:** Snowflake's built-in catalog, implementing the open Apache Iceberg REST Catalog and Apache Polaris™ APIs natively in every Snowflake account. Any engine implementing the open Iceberg REST specification can connect without Snowflake-specific integration work, with governance policies enforced at the catalog layer for all connected engines.

Your business catalog is not your Iceberg catalog

There are two distinct catalog layers in a modern data platform. Conflating them may lead to less optimal architectural decisions.

Layer	Examples	What it does
Iceberg table catalog	Snowflake Horizon, Databricks UC, Apache Polaris, AWS Glue	Manages table metadata, snapshot history, schema and data file locations. Compute engines use this to read and write data.
Business / governance catalog	Alation, Informatica CDGC, Collibra	Business glossary, data discovery, lineage documentation. Not operational metastores — compute engines cannot use them to access data.

This post is about **layer one**: Which operational Iceberg catalog should own your data.

Five decision principles

Before evaluating any specific catalog, define the criteria that matter for your architecture. These five principles should guide your evaluation:

1. Open standard

Does the catalog implement the **open Apache Iceberg REST Catalog specification**? Or does it use vendor-specific protocols that may limit which engines can connect and what operations they can perform?

An open-standard catalog means any engine that supports the Iceberg REST protocol — Apache Spark™, Apache Flink®, Trino, DuckDB, Pylceberg and more — can connect natively. A proprietary catalog may limit your options to integrations the vendor builds and maintains.

UC's tighter integration within the Databricks ecosystem means engines running on Databricks get optimized metadata access without needing an external REST protocol. However, for organizations operating across multiple engines, an open-standard catalog provides native connectivity without vendor-specific adapters.

Both Horizon and Unity Catalog support credential-vending concepts — issuing temporary, scoped storage credentials to connecting engines. However, they differ in external-engine support, governance model and write-path maturity:

- Horizon implements the open Apache Iceberg REST Catalog specification (Apache Polaris). Any engine supporting the standard protocol can connect, receive vended credentials, and perform both reads and writes — with governance policies enforced at the catalog layer regardless of engine.
- Unity Catalog supports credential vending through its own REST APIs, but external-engine support is more limited in scope. Non-Databricks engines can connect via Lakehouse Federation (JDBC, read-only) or through UC's credential-vending endpoints, though write-path support and governance enforcement for external engines varies by integration path and is less established than the within-Databricks experience.

2. Governance universality

Are access policies enforced for **every engine** that connects, or only within one platform?

Within Databricks, UC provides mature, unified governance features — fine-grained access control, row/column-level security and automatic lineage tracking — across all Databricks workloads without additional configuration.

When a governance framework is enforced at the catalog layer, policies apply uniformly regardless of which engine connects. When policies are enforced separately across different systems or access paths, organizations must ensure consistency across each integration point — which is achievable but increases operational complexity and the surface area for policy drift.

3. Bidirectional interoperability

Two directions matter here:

- Inbound: Can external engines read and write to tables your catalog manages?
- Outbound: Can your catalog's engine read and write to tables managed by other catalogs?

True bidirectionality means both directions work. Snowflake can read and write to UC-managed and Glue-managed Iceberg tables via catalog-linked databases — in addition to exposing Horizon-managed tables to external engines via the open Iceberg REST protocol. When a catalog's external-access paths offer different capabilities depending on the integration method, some engines end up with full read/write access while others are limited to read-only or require vendor-specific connectors. Evaluate whether both directions are supported with full capability, or just one.

4. Operational independence

Does choosing this catalog create a **runtime dependency on another platform's infrastructure**?

For Databricks-primary organizations, UC consolidates catalog operations within existing infrastructure — no additional platform dependency is introduced. However, for multi-platform environments, if your Snowflake workloads depend on Unity Catalog availability — or vice versa — you've introduced a cross-platform dependency you don't fully control. Evaluate whether a catalog outage on one platform affects workloads running on the other.

5. Feature completeness

Does choosing this catalog **unlock or restrict** your platform's native capabilities?

UC-owned tables unlock Databricks-native capabilities: Unity Catalog lineage, ML model and feature store governance, and Delta Sharing for cross-organization data exchange.

Horizon-owned tables unlock Snowflake-native capabilities: dynamic tables, Cortex AI functions, secure data sharing and marketplace publishing. A catalog choice shouldn't force you to give up features you've already invested in — evaluate which platform's feature set matters more for each workload, and whether the catalog choice restricts the other platform's capabilities.

The core differentiator: Community-driven vs. single-vendor open source

Both Snowflake and Databricks have open source catalog projects — Apache Polaris (an open source project governed by the Apache Software Foundation) for Snowflake and Unity Catalog OSS (Linux Foundation) for Databricks. However, they differ in governance model and relationship to the managed product.

Apache Polaris (the foundation of Horizon Catalog) is an ASF project governed by Apache Software Foundation processes — including community-driven development, open governance and a requirement for diverse project oversight independent of any single contributor. The project focuses on Iceberg-centric problems — credential vending, multi-engine access, catalog federation — driven by community needs.

Unity Catalog OSS is a Databricks-led open source release. Databricks controls the project direction and decides which capabilities exist in the OSS version vs. the managed offering.

Horizon is a Snowflake product, but it implements the open Apache Iceberg REST specification governed by ASF. The interface to your data is an open standard — if you ever move away from Snowflake, any engine supporting the Iceberg REST protocol can still access your tables without Snowflake-specific integration.

How catalog ownership works

When an engine writes an Iceberg table, it registers the table with a catalog. That catalog becomes the **source of truth** for:

- Table metadata (schema, snapshots, partition spec)
- Data file locations
- Access control enforcement (who can read/write)
- Table maintenance (compaction, vacuum, optimization)

The catalog that **manages** the table governs it. Engines that connect to that catalog are subject to the catalog's access policies — not their own. This is why the choice matters: The catalog owner controls the rules for everyone.

Applying the principles: A quick summary

Principle	Question to ask	Snowflake Horizon as catalog of record	Databricks Unity Catalog as catalog of record
Multi-engine access	How do non-native engines connect?	Via Iceberg REST Catalog spec with vended credentials (Apache Spark™, Apache Flink®, DuckDB, Pylceberg, etc.)	Via Lakehouse Federation (JDBC, read-only for non-Databricks engines)
Open standard governance (Iceberg REST)	Who controls the catalog's roadmap?	ASF community (multi-vendor, Apache Polaris); same engine as Horizon's managed service; multi-vendor governance	Databricks UC (single-vendor OSS under Linux Foundation) — different codebase from managed UC, single-vendor governance
Governance enforcement	Where are access policies enforced?	At the catalog layer: applies to all connecting engines via vended credentials	At the UC layer — applies to all connecting engines, including Snowflake via CLD
Bidirectional access	Can you read AND write across catalogs?	Yes, Snowflake reads/writes to UC and Glue-managed tables (GA); external engines read/write to Horizon-managed tables (preview)	Read-only: Lakehouse Federation is JDBC, read-only for external catalogs. No write-back to Horizon or Glue-managed tables.
Platform features	Which features are unavailable if this isn't your native catalog?	N/A — full Snowflake feature set (Dynamic Tables, Cortex AI, Data Sharing)	Dynamic Tables, Cortex AI and Data Sharing/Marketplace are unavailable on UC-managed tables in Snowflake

What's next

In [Part 2](#), we apply these principles in detail — examining the full read/write capability matrix across engines, what "bidirectional" actually means in practice, how governance enforcement works at the catalog layer, and the specific features you gain or lose with each choice.

[Read Part 2: Snowflake Horizon vs. Databricks Unity Catalog: The Technical Comparison](#)

Learn more about the author



Richie Bachala

Sr. Manager, Solution Architecture



Share this post



Subscribe to our blog newsletter

Get the best, coolest and latest delivered to your inbox each week

Email Address

SUBSCRIBE NOW

By submitting this form, I understand Snowflake will process my personal information in accordance with their Privacy Notice.



WHERE DATA DOES MORE

START FOR FREE

WATCH A DEMO

Product

Platform

Snowflake CoWork

Data Engineering

Analytics

AI

Applications & Collaboration

Pricing

Support

Support

Priority Support

Status [↗](#)

Company

About Snowflake

Leadership & Board

Careers [↗](#)

Investor Relations [↗](#)

Trust Center [↗](#)

Brand Guidelines [↗](#)

Contact

Newsroom

Environmental, Social & Governance

Snowflake Ventures

End Data Disparity

Snowflake Summit 26

Industries

Advertising, Media & Entertainment

Financial Services

Healthcare & Life Sciences

Manufacturing

Public Sector

Retail & Consumer Goods

Telecom

Technology

Learn

Resource Library

Live Demos

Fundamentals

Training

Certifications

Snowflake University [↗](#)

Developer Guides

Documentation [↗](#)

Data Governance

Subscribe to our monthly newsletter

Stay up to date on Snowflake's latest products, expert insights and resources—right in your inbox!

☐

Add me to the list to receive dedicated product updates and general availability emails.

By submitting this form, I understand Snowflake will process my personal information in accordance with its [Privacy Notice](#). I may unsubscribe through [unsubscribe links](#) at any time.

SUBSCRIBE NOW



© 2026 Snowflake Inc. All Rights Reserved

[Privacy Policy](#)

[Site Terms](#)

[Communication Preferences](#)

[Cookies Settings](#)

[Do Not Share My Personal Information](#)

[Legal](#)

